FULL-STACK DEVELOPER PORTFOLIO

Portfolio

여섯 개의 프로젝트를 통한 성장 기록

PROJECT 01
AuRa
향수 이커머스
4인 팀 · 3주

PROJECT 02
KISETSU
도매 패션 이커머스
1인 솔로 · 2주

PROJECT 03
재무제표 자동화
Python + BrityRPA
3인 팀 · 2주

PROJECT 04
개발환기좀해 ERP
Spring Boot + JSP
1인 솔로 · 5주

PROJECT 05
FIRST 팀 쇼핑몰
React + Node.js
4인 팀 · 3주

PROJECT 06
계절 솔로 쇼핑몰
React + Socket.io
1인 솔로 · 10일

React · Node.js · Spring Boot · MariaDB · JWT · Socket.io · MyBatis · Python · BrityRPA · Nodemailer

2025 — 2026

INDEX

목차

PROJECT 01 — AuRa 향수 쇼핑몰

01 **프로젝트 개요**
AuRa 소개, 기술 스택, 팀 구성 — 03

02 **주요 기여 — DB · 인증 · 장바구니 · 카테고리**
담당 기능 상세, 코드 구현 방식 — 04

03 **기술적 어필 포인트**
관심사 분리, 중복 방지 로직, 서버단 검증 — 05

PROJECT 02 — KISETSU 도매 패션 이커머스

04 **프로젝트 개요 · 아키텍처**
기획 배경, 시스템 구조, 기술 스택 — 07

05 **주요 기능 구현**
JWT 인증, 실시간 채팅, Context API, 인기검색어 — 09

06 **결과 · 회고**
구현 성과, 배운 점, 성장 스토리 — 10

PROJECT 03 — 재무제표 자동화 (Python + BrityRPA)

07 **프로젝트 개요**
기획 배경, 시스템 아키텍처, 담당 역할 — 12

08 **담당 역할 — RPA 자동화 파이프라인**
아키텍처 설계, RPA 구현, 데이터 검증 — 13

09 **핵심 기능 구현**
Python 실행, Excel 차트 자동화, 메일 발송 — 14

10 **결과 · 회고**
RPA 성과, 잘된 점, 아쉬운 점, 자체 평가 — 15

PROJECT 04 — 개발환기좀해 ERP (Spring Boot + MyBatis)

11 **프로젝트 개요 · 아키텍처**
기획 배경, 전체 모듈, 기술 스택 — 17

12 **핵심 기능 구현**
OCR 거래명세서, BOM 자동발주, 재무 리포트 — 18

13 **성능 최적화 · DB 쿼리 튜닝**
N+1 문제 해결, 인덱스 최적화 (14.25s→0.17s) — 19

14 **기술적 어필 & 회고**
Spring Security, 트랜잭션, 성과, 회고 — 20

PROJECT 05 — FIRST 팀 쇼핑몰 (React + Node.js)

15 **프로젝트 개요 · 기술스택**
팀 구성, React + Express + MariaDB, 담당 파트 — 21

16 **고난과 역경 — Git이 증명하는 개발 여정**
"결제된다 무야호" · 머지 충돌 · IP 전쟁 — 22

17 **해결 & 회고**
결제 구현 돌파, 협업 충돌 극복, 성장 스토리 — 23

PROJECT 06 — 계절 솔로 쇼핑몰 (React + Node.js + Socket.io)

18 **프로젝트 개요 · 10일 여정**
솔로 풀스택, JWT · Socket.io · Chart.js · Nodemailer — 24

19 **고난과 역경 — 10일간의 기록**
환경 불일치 · IP 전쟁 · 채팅 스크롤 사투 — 25

20 **결과 & 회고**
10일 완성, 전체 기능 솔로 구현, 성장 포인트 — 26

PROJECT 01

AuRa

향수 이커머스 쇼핑몰
REST API · 풀스택 팀 프로젝트

유형	기간	역할
4인 팀 프로젝트	약 3주	DB · 인증 · 장바구니 · 카테고리

MariaDB · Node.js · Express · React · Vite · REST API · 커넥션 풀

01

프로젝트 개요

AuRa 향수 쇼핑몰 — 팀 프로젝트

4인	3주	4개	REST
팀 구성	개발 기간	담당 기능	API 설계

서비스 전체 기능

- 회원가입 / 로그인
- 상품 목록 / 상세 / 검색
- 카테고리 필터 & 정렬
- 장바구니 / 위시리스트
- 주문 / 결제 처리
- AI 챗봇 상담
- 미니게임 이벤트

내 담당 기능

- **DB 설계** — MariaDB 커넥션 풀 모듈화
- **로그인 / 회원가입** — REST API, 서버단 검증
- **장바구니** — CRUD API, 중복 방지 로직
- **카테고리** — DB 기반 동적 카테고리, 필터 & 정렬

기술 스택

Database	Backend	Frontend
MariaDB	Node.js, Express	React, Vite

02 주요 기여 — 구현 상세

DB 설계 · 로그인/회원가입 · 장바구니 · 카테고리

DB — 커넥션 풀 모듈화

db.js

```
// MariaDB 커넥션 풀 모듈 분리
const mariadb = require('mariadb')
require('dotenv').config()

const pool = mariadb.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  connectionLimit: 10
})

module.exports = pool
```

로그인 / 회원가입 API

auth.js

```
// POST /api/auth/login
app.post("/api/auth/login", async (req, res) => {
  const { username, password } = req.body
  // 입력값 누락 시 서버단 검증
  if (!username || !password)
    return res.status(400).json({ msg: "필수값 누락" })
  const [user] = await db.query(
    `SELECT * FROM users
     WHERE username=? AND password=?`,
    [username, password]
  )
  if (!user) return res.status(401).json({...})
  res.json({ success: true, user })
})
```

장바구니 — 중복 방지 핵심 로직

cart.js

```
// POST /api/cart/add
// 핵심: 중복 상품 서버단 검증
app.post("/api/cart/add", async (req, res) => {
  const { userId, productId } = req.body
  const [exist] = await db.query(
    `SELECT * FROM cart
     WHERE user_id=? AND product_id=?`,
    [userId, productId]
  )
  if (exist)
    return res.json({ msg: "이미 담긴 상품" })
  await db.query(
    `INSERT INTO cart VALUES(?,?)`,
    [userId, productId]
  )
})
```

카테고리 — DB 기반 동적 쿼리

category.js

```
// GET /api/category (DB 기반)
app.get("/api/category", async (req, res) => {
  // 하드코딩 없이 DB에서 동적 조회
  const cats = await db.query(
    `SELECT * FROM categories`
  )
  res.json(cats)
})
// 가격 필터 + 정렬 동적 쿼리
let sql = `SELECT * FROM products
  WHERE category_id=?`
if (minPrice) sql += ` AND price≥?`
sql += ` ORDER BY \${sortField}`
```

03

기술적 어필 & 회고

설계 철학과 팀 프로젝트에서의 성장

관심사 분리 (모듈화)

DB 연결 로직을 `db.js` 로 분리해 전체 서버에서 재사용 가능하도록 설계. 유지보수 시 DB 설정 변경이 단 1개 파일만으로 가능.

서버단 중복 방지 로직

장바구니·위시리스트 추가 시 프론트가 아닌 서버단에서 DB 조회 후 검증. 클라이언트 조작에도 데이터 무결성 보장.

동적 카테고리 설계

카테고리를 하드코딩이 아닌 DB 기반으로 관리해 코드 수정 없이 카테고리 추가/삭제 가능. 가격 필터와 정렬도 동적 쿼리로 지원.

.env 보안 분리

DB 접속 정보를 `.env` 로 분리해 소스 코드에 직접 노출되지 않도록 보안 강화. 커넥션 풀로 DB 성능 최적화.

팀 프로젝트에서 배운 것

첫 팀 프로젝트였던 만큼 기술적인 어려움뿐 아니라 협업 자체의 어려움도 크게 느꼈습니다. 초반에는 각자 개발한 기능이 서로 맞지 않거나, 말로는 이해한 것 같았지만 실제 구현 결과가 달라 수정이 반복되는 상황이 생겼습니다. **의사소통이 충분하지 않으면 개발 속도보다 수정 비용이 더 커진다**는 것을 몸소 경험했습니다. 이 경험은 이후 KISETSU 솔로 프로젝트에서 설계를 철저히 하는 원동력이 되었습니다.

PROJECT 02

KISETSU

도매 패션 이커머스 플랫폼
실제 운영자 요구 기반 · 1인 풀스택 솔로 프로젝트

유형	기간	규모
1인 솔로 프로젝트	2026.01.15 – 01.30 (2주)	고객 기능 8개 + 관리자 기능 8개

React 19 · Node.js · Express 5 · MariaDB · JWT · Socket.IO · Chart.js · bcrypt · Context API

04

프로젝트 개요 · 아키텍처

KISETSU — 실제 도매업 운영자의 요구를 반영한 플랫폼

개발 목적

- 실제 도매업 운영자의 **온라인 판매 채널 부재 문제** 해결
- 재고·주문·고객 관리를 **하나의 플랫폼으로 통합**
- 오프라인 의존도를 낮추고 **운영 효율화** 도모
- 운영자(지인)와 직접 대화하며 요구사항 수집

주요 기능 목록

- 고객: 회원가입/로그인(JWT) · 상품목록 · 장바구니 → 주문
- 고객: 자동완성 검색 + 인기검색어
- 고객: 실시간 1:1 채팅 (Socket.IO)
- 고객: 계절별 테마 UI (봄/여름/가을/겨울)
- 관리자: 대시보드 · 상품/주문/회원 관리
- 관리자: Chart.js 매출 통계

시스템 아키텍처

Frontend →	React 19 + Vite	고객 화면	관리자 화면
통신 →	HTTP (Axios) REST API	WebSocket (Socket.IO)	
Backend →	Node.js + Express 5	/auth · /products · /orders · /admin · /chat	
Auth →	JWT 미들웨어	bcrypt 해싱	역할 기반 접근제어
Database →	MariaDB	커넥션 풀	트랜잭션 처리

05

주요 기능 구현

JWT 인증 · Socket.IO 채팅 · Context API · 인기검색어

JWT 인증 미들웨어

authMiddleware.js

```
const authMiddleware = (req, res, next) => {
  try {
    const token = req.headers
      .authorization?.split(" ")[1]
    if (!token)
      return res.status(401).json({
        msg: "인증 토큰 없음"
      })
    const decoded = jwt.verify(
      token, process.env.JWT_SECRET
    )
    req.user = decoded
    next()
  } catch {
    res.status(403).json({ msg: "유효하지 않은 토큰" })
  }
}
```

SOCKET.IO 실시간 채팅

Chat.jsx

```
// 1단계: 채팅 열릴 때 소켓 연결
useEffect(() => {
  if (isOpen && isLoggedIn) {
    const newSocket = io(SOCKET_URL)
    setSocket(newSocket)
    return () => newSocket.disconnect()
  }
}, [isOpen, isLoggedIn])

// 2단계: 소켓 생기면 이벤트 구독
useEffect(() => {
  if (!socket) return
  socket.on('message', (msg) => {
    setMessages(prev => [...prev, msg])
  })
  return () => socket.off('message')
}, [socket])
```

인기검색어 — SQL 비즈니스 로직

search.js

```
// 최근 7일 주문 기반 인기 상품 추출
const [popular] = await db.query(`
  SELECT p.name,
    COUNT(oi.order_item_id) as order_count
  FROM products p
  JOIN order_items oi ON p.id = oi.product_id
  JOIN orders o ON oi.order_id = o.id
  WHERE o.created_at ≥
    DATE_SUB(NOW(), INTERVAL 7 DAY)
  GROUP BY p.id
  ORDER BY order_count DESC
  LIMIT 5
`)
```

CONTEXT API — 장바구니 전역 상태

CartContext.jsx

```
const CartContext = createContext()

// 커스텀 훅으로 사용처 보호
export const useCart = () => {
  const ctx = useContext(CartContext)
  if (!ctx)
    throw new Error(
      "CartProvider 밖에서 호출됨"
    )
  return ctx
}

export const CartProvider = ({ children }) => {
  const [cart, setCart] = useState([])
  return (
    <CartContext.Provider value={{ cart, setCart }}>
      {children}
    </CartContext.Provider>
  )
}
```

06

결과 · 종합 회고

KISETSU 성과와 두 프로젝트를 통한 성장

8개	8개	100h	실전
고객 기능	관리자 기능	2주 단독 완성	운영자 요구 기반

잘된 점

- 첫 프로젝트 대비 코드 구조와 흐름이 읽히기 시작
- 기획부터 구현까지 전 과정을 혼자 완성
- 실제 운영자 요구를 반영한 서비스 설계 경험
- JWT, Socket.IO, Context API 등 실무 기술 적용
- 풀스택 데이터 흐름 전반 이해

아쉬운 점 / 다음 목표

- 테스트 코드 부재 — 다음 프로젝트에서 도입 목표
- 리팩토링 시간 부족으로 중복 코드 일부 존재
- TypeScript 도입으로 타입 안정성 확보 계획
- CI/CD 파이프라인 구성 학습 중

성장 스토리 — AuRa → KISETSU

AuRa 팀 프로젝트에서 협업의 어려움과 기술적 한계를 경험한 후, KISETSU에서는 **설계를 먼저, 구현은 그 다음**이라는 원칙을 철저히 지켰습니다. DB 테이블 설계, REST API 명세, 컴포넌트 구조를 사전에 정리한 덕분에 2주 안에 고객·관리자 기능 총 16개를 단독으로 완성할 수 있었습니다. 단순히 작동하는 코드를 넘어, **왜 이렇게 설계했는가를 설명할 수 있는 개발자**로 성장하는 것이 목표입니다.

PROJECT 03

재무제표 자동화

Python + BrityRPA · 3인 팀 프로젝트

유형	기간	역할	파이프라인
3인 팀 프로젝트	**26.02.28 – 03.10**	**팀장 · RPA 개발**	**End-to-End 자동화**

Python · BrityRPA · Pandas · openpyxl · SMTP · K-IFRS

07

프로젝트 개요

재무제표구현 — Python + BrityRPA 자동화 팀 프로젝트

3인	2주	5개	100%
팀 구성	개발 기간	RPA 자동화	Zero-Touch 파이프라인

개발 목적

기업 재무팀의 수작업 회계 전표 처리 자동화. 매월 수만 건의 Raw Data를 수작업으로 처리하는 비효율과 휴먼 에러 제거.

프로젝트 목표

- Python → 재무 데이터 처리 (ETL)
- RPA → 보고서 자동 생성 & 차트
- 메일 자동 발송 (SMTP)
- Raw Data → 재무제표 → 대시보드 → 메일

시스템 아키텍처

Input CSV Raw Data (1만 건 가상 분개장)

Process Python · Pandas ETL → 데이터 마트

Output BrityRPA → Excel 차트 · 메일 발송

내 담당 역할 (팀장 · RPA 개발)

End-to-End 자동화 아키텍처 설계

· Python + BrityRPA 연계 자동화 구조 설계
· ETL → 데이터마트 → Excel 전체 파이프라인 구축

RPA 자동화 구현

· Python 스크립트 실행 및 결과 수집 자동화
· Excel 보고서 생성 / 그래프 / 레이아웃 자동 구성

데이터 검증 및 프로젝트 관리

· 재무 데이터 정합성 검증 UI 기반 Validation 설계
· 프로젝트 일정 관리 및 기술 의사결정 리딩

기술 스택

Data Engineering & Logic: Python, Pandas, openpyxl

Process Automation: BrityRPA, SMTP

Data Source: CSV Raw Data, K-IFRS 기준

08

담당 역할 — RPA 자동화 파이프라인

팀장으로서 End-to-End 자동화 아키텍처 설계 및 BrityRPA 구현 전담

CSV Raw Data → **Python** ETL → **Data** Mart → **BrityRPA** 자동화 → **Excel** 보고서 → **메일** 발송

① 아키텍처 설계

Hybrid Automation 설계

Python이 연산, RPA가 시각화/배포를 전담하는 역할 분리 구조

인터페이스 규격 수립

Python ↔ RPA 간 데이터 교환 형식 사전 합의. error_flag.txt로 오류 상태 공유

Error Handling 구조

Python 실행 중 오류 발생 시 플래그 파일 생성 → RPA 자동 중단 설계

전체 파이프라인 리딩

ETL → 데이터마트 → Excel 리포트까지 전체 흐름 총괄

② RPA 자동화 구현

Python 스크립트 실행

BrityRPA ExecuteCmd로 apnalchangchang.py 실행 및 결과 수집

Excel 차트 자동 생성

월별 손익 꺾은선 차트 / 비용구조 파이차트 / 포괄손익계산서 차트

분기별 재무현황 이관

Python 결과 파일 → 보고서 템플릿 시트 자동 복사 · 서식 유지

엑셀 저장 · 메일 발송

SMTP 연동, 수신자 자동 설정, 완성된 Excel 보고서 자동 첨부 발송

③ 데이터 검증 & 관리

재무 데이터 정합성 검증

UI 기반 Validation 설계로 입력 데이터 오류 사전 차단

프로젝트 일정 관리

공통 기획 → Python 개발 → RPA 시각화 → 통합 테스트 단계 리딩

기술 의사결정

Python-RPA 연계 방식, 데이터 마트 구조, 오류 플래그 방식 등 결정

팀 협업 조율

타 기술 스택 담당자와 명확한 데이터 인터페이스 합의 및 소통

09

핵심 기능 구현

BrityRPA 기반 자동화 · Python-RPA Hybrid Architecture

1. 파이썬 실행 자동화

BrityRPA_ExecuteCmd.bot

BrityRPA의 ExecuteCmd 액션으로 Python 데이터 처리 엔진 실행. RPA가 단순 UI 자동화를 넘어 전체 자동화 파이프라인을 제어하는 Orchestrator 역할 수행.

```
// 1. Python 스크립트 실행
ExecuteCmd("python apnalchangchang.py")

// 2. 오류 플래그 감지
IF FileExists("error_flag.txt")
  → RPA 프로세스 중단 처리
```

2. Excel 차트 자동 생성

chart_automation.bot

Python 결과 데이터 기반 Excel Chart Object 자동 생성. 월별 손익 꺾은선, 비용구조 파이차트, 포괄손익계산서 차트 3종 자동화.

```
// 차트 자동화 프로세스
CreateChart(type="Line")
  .SetDataRange(monthly_pl)
  .SetTitle("월별 손익 추이")
CreateChart(type="Pie")
  .SetDataRange(cost_structure)
```

3. 분기별 재무현황 이관

data_transfer.bot

Python 생성 재무현황 데이터를 최종 보고서 Excel 템플릿으로 자동 이관. 기존 서식·레이아웃 유지한 상태로 데이터 배치.

```
// 데이터 이관 프로세스
LoadWorkbook(data_mart.xlsx)
SelectSheet("분기별재무현황")
CopyRange(A1:Z100)
PasteToTemplate("보고서템플릿")
MaintainFormatting = True
```

4. 엑셀 저장 & 메일 자동 발송

mail_dispatch.bot

SMTP 서버 연동으로 최종 보고서 자동 배포. 수신자 자동 설정, 메일 제목·본문 자동 작성, Excel 첨부 발송까지 완전 자동화.

```
// End-to-End 메일 발송
SaveWorkbook("경영대시보드.xlsx")
SMTP.Connect(server, port)
Mail.SetRecipients(recipients)
Mail.AttachFile(report.xlsx)
Mail.Send()  // 자동 발송 완료
```

10

결과 · 회고

RPA 자동화 프로젝트 성과와 팀장으로서의 성장

5개	100%	1만 건+	팀장
RPA 자동화 구현	Zero-Touch 파이프라인	전표 데이터 처리	E2E 아키텍처 리딩

잘된 점

- 재무자동화 핵심 프로세스 100% 구현
- Python-RPA 하이브리드 아키텍처 완성
- Error Flag 구조로 이기종 시스템 안정 연동
- 원천데이터 파싱 → 메일 발송 Zero-Touch 완성
- 팀장으로 전체 기술 의사결정 및 일정 리딩

△ 아쉬운 점 / 개선 방향

- 초기 원천 데이터 예외 포맷 오류 처리 미흡
- RPA 예외처리 로직 더 정교화 필요
- 동적 방어 코드 추가 및 에러 로깅 고도화 목표
- 실제 경영진 피드백 기반 대시보드 UI/UX 개선

자체 평가 — 윤종빈

단순한 스크립트 코딩을 넘어, 실제 기업 재무 기준을 시스템 로직으로 완벽히 구현해낸 것이 이번 프로젝트의 가장 큰 성과입니다. Python이 무거운 "데이터 연산"을 전담하고 RPA가 "시각화 및 메일 발송"을 담당하도록 아키텍처를 분리하였습니다. 다른 기술 스택을 담당하는 팀원과 명확한 데이터 인터페이스 규격을 합의하고, 오류 발생 시 즉각적으로 상태를 공유하는 Error Flag 구조를 설계하며 이기종 시스템 간 매끄러운 연동과 소통 기반의 협업 시너지를 제대로 경험할 수 있었습니다.

PROJECT 04

개발환기좀해 ERP

Spring Boot + MyBatis · 1인 솔로 풀스택

유형	기간	규모	특이사항
1인 솔로 프로젝트	**26.03.25 – 04.01**	**모듈 15개+**	**OCR · AI API 연동**

Spring Boot 3.4 · Java 17 · MyBatis · MariaDB · Spring Security · JSP/JSTL · Swagger · Gemini OCR · Lombok

11

프로젝트 개요 · 전체 모듈

개발환기좀해 ERP — 제조업 기반 통합 자원 관리 시스템 (1인 풀스택)

15개+	20개+	1인	5주
컨트롤러/모듈	REST API	솔로 개발	개발 기간

인증 · 보안
Spring Security 역할 기반 접근제어
BCrypt 패스워드 해싱
ADMIN / USER 권한 분리

대시보드
실시간 KPI (제품수, 직원수, 매출)
재고부족 알림, 월별 작업지시 통계
최근 주문 · 저재고 품목 리스트

인사 · 근태
직원 CRUD + 사진 업로드
출퇴근 체크인/아웃 API
연차/병가 등록 · 잔여일수 차감

생산 관리
BOM(자재명세서) 관리
작업지시서 생성 · 상태 관리
BOM 기반 재고 부족 시 자동 발주

재고 · 창고
재고 CRUD, 입출고 트랜잭션
창고별 재고 현황 관리
저재고 기준치 설정 (시스템 설정)

구매 · 영업
발주서 워크플로우 (대기→승인→완료)
판매 · 배송 관리
거래처(공급업체) 관리

거래명세서
OCR (Gemini/Groq AI) 자동 인식
이미지 업로드 → DB 자동 등록
인쇄 기능 (거래명세서 출력)

재무 분석
손익계산서 자동 계산 (월별/연별)
영업이익률 · 매출총이익 산출
Python 연동 Excel 리포트 내보내기

관리자
사용자 계정 관리 (ADMIN)
저재고 기준치 시스템 설정
Swagger UI API 문서 자동화

12

핵심 기능 구현

OCR 자동 인식 · BOM 자동발주 · 재무 리포트 · Spring Security

OCR 거래명세서 자동 인식

OcrController.java

Gemini/Groq AI API 연동. 이미지 업로드 → Python OCR 실행 → JSON 파싱 → DB 자동 저장까지 완전 자동화.

```
// 1. 이미지 → Python OCR 실행
ProcessBuilder pb = new ProcessBuilder(
  pythonCmd, mainPy, "--erp-mode", file);
// 2. JSON 파싱 → TransactionStatement
stmt.setIssueDate(LocalDate.parse(...));
statementService.insert(stmt, items);
```

BOM 기반 자동 부품 발주

WorkOrderService.java

작업지시 시 BOM 조회 → 재고 부족 부품 자동 감지 → 발주서 자동 생성. 부품 원가 변경 시 완제품 원가 BOM 기반 자동 재계산.

```
// 재고 부족 부품 자동 발주
List<BomItem> shortages =
  bomService.checkShortages(workOrderId);
for (BomItem item : shortages) {
  purchaseOrderService.autoCreate(item);
} // 발주서 자동 생성
```

재무 분석 · Python Excel 리포트

FinancialController.java

매출/비용 데이터 기반 손익계산서 자동 산출. Python 스크립트 연동으로 xlsx 리포트 생성 및 다운로드.

```
// 영업이익률 자동 계산
BigDecimal opMargin = opProfit
  .divide(revenue, 4, HALF_UP)
  .multiply(BigDecimal.valueOf(100));
// Python → xlsx 리포트 반환
byte[] xlsx = runPythonClean(csvBytes);
```

Spring Security 역할 기반 인증

SecurityConfig.java

BCrypt 해싱, 폼 로그인, ADMIN/USER 역할 분리. /admin/** 경로는 ADMIN만 접근. JSP Forward/Include 요청 허용 처리.

```
// 역할 기반 접근 제어
.requestMatchers("/admin/**")
  .hasRole("ADMIN")
.anyRequest().authenticated()
// BCrypt 패스워드 인코더
return new BCryptPasswordEncoder();
```

13

성능 최적화 · DB 쿼리 튜닝

BEFORE
14.25 s → AFTER **0.17 s (176 ms)** **83x 향상**

① N+1 쿼리 문제 해결

BEFORE

목록 각 행마다 SUM() 서브쿼리 반복 실행
→ 데이터 100건 = SQL 101번 호출 (과부하)

```
SELECT *, (SELECT SUM(...)
  FROM items WHERE order_id = o.id)
FROM orders o  -- 행마다 재실행!
```

AFTER

파생 테이블로 SUM 미리 집계 후 1회 JOIN
→ 데이터 100건도 SQL 1번으로 처리

```
SELECT o.*, s.total FROM orders o
LEFT JOIN (SELECT order_id,
  SUM(qty) total FROM items
  GROUP BY order_id) s ON o.id=s.order_id
```

② JOIN 조건 · 인덱스 최적화

BEFORE

LIKE 연산자로 customer_name 매칭
→ 인덱스 무효화, 풀 스캔 발생

```
JOIN suppliers s
  ON o.supplier_info
     LIKE CONCAT('%',s.name,'%')
-- 전체 스캔으로 매우 느림
```

AFTER

customer_name 컬럼 직접 참조로 변경
→ 인덱스 즉시 활용, 속도 대폭 향상

```
JOIN suppliers s
  ON o.customer_name = s.name
-- 인덱스 활용 → 즉시 탐색
-- 14.25s → 0.17s 달성
```

배운 점

"기능 구현"에서 끝나지 않고 실제 사용 환경에서 병목을 직접 발견하고 쿼리 구조를 재설계하여 83배의 성능 향상을 달성했습니다. N+1 문제와 인덱스 활용의 중요성을 실무 수준으로 체득한 경험입니다.

14

기술적 어필 & 회고

개발환기좀해 ERP — 1인 풀스택 완성 성과

15개+	100%	OCR	트랜잭션
모듈 구현	1인 완성	AI 연동	원자성 보장

기술적 어필 포인트

- OCR + AI API (Gemini/Groq) 연동 거래명세서 자동 등록
- BOM 기반 재고 부족 자동 감지 → 발주서 즉시 생성
- 입고/출고/생산입고 @Transactional 원자성 처리
- Spring Security 역할 기반 접근제어 + BCrypt
- Python 연동 재무 Excel 리포트 (csv → xlsx 변환)
- 사이드바 아코디언 UI + 현재 메뉴 자동 펼침

△ 아쉬운 점 / 개선 방향

- JSP + JSTL 한계 → 다음엔 React 프론트 분리 목표
- OCR 예외 포맷(누락 · 오타) 처리 로직 보완 필요
- 테스트 코드 없음 → JUnit + MockMvc 도입 계획
- 실시간 알림(WebSocket) 추후 적용 예정

회고 — 개발환기좀해 ERP

제조업 ERP 전 모듈을 1인으로 처음부터 끝까지 구현하면서, 단순한 CRUD를 넘어 실제 업무 프로세스를 코드로 표현하는 경험을 했습니다. BOM 기반 자동 발주, OCR AI 연동, Python 재무 리포트 등 서로 다른 기술 스택을 하나의 시스템으로 통합하는 과정에서 Spring Boot의 계층 구조(Controller-Service-Mapper)와 트랜잭션 관리의 중요성을 몸소 익혔습니다. JSP의 한계를 느끼면서 React 분리의 필요성도 깨달았고, 다음 프로젝트에서는 프론트엔드를 완전히 분리한 REST API 기반 아키텍처를 목표로 합니다.

PROJECT 05

FIRST 팀 쇼핑몰

React + Node.js · 4인 팀 프로젝트 · 3주

유형	기간	역할	팀 저장소
4인 팀 프로젝트	25.12.02 – 12.18	결제·관리자	kyeol1202/Team4-Project

React 18 · Node.js · Express 5 · MySQL2 · MariaDB · OpenAI · Multer · React Router

결제 시스템
카드 결제 흐름 구현 (주문 생성 → 결제 처리 → 성공/실패 분기)

관리자 페이지
상품/주문/회원 CRUD, 관리자 로그인 JWT 인증

AI 챗봇
OpenAI API 연동 상품 추천 & Q&A 자동 응답

위시리스트 · 장바구니
Context API 전역 상태, DB 연동 위시/장바구니

교환 · 반품
반품/환불 접수 폼, 마이페이지 주문 내역 연동

검색 · 카테고리
상품 검색, 카테고리별 필터링, 상품 상세

16

고난과 역경

Git이 증명하는 개발 여정 — 실제 커밋 메시지 기록

Dec 10

관리자용수정중인데모르겟서요엉엉

어드민 페이지 구조를 어떻게 설계해야 할지 막막했던 순간. 권한 분리, JWT 인증 경로, 라우팅 구조 전부 처음이었음.

Dec 12

진짜진짜진짜

결제 로직이 계속 오류. 주문 생성 → 결제 연결 → DB 저장까지 흐름이 연결이 안 됨. 커밋 이름이 절박함을 표현.

Dec 12

결제된다 무야호

드디어 결제 흐름 완성! /api/order/create 라우트와 프론트 Payment 컴포넌트가 맞물림. 팀 최고 명장면.

Dec ~

장바구니 디자인 복구

develop 브랜치 머지 이후 cart-pay.css가 충돌로 깨짐. 308줄 삭제하고 재설계. 머지 충돌의 교훈.

Dec 4

ip변경 (203 → 224)

DB_HOST가 컴퓨터마다 달라 .env를 계속 수정. 팀 공통 DB 서버 IP를 통일하는 과정.

17

해결 & 회고

FIRST 팀 쇼핑몰 — 협업 충돌 극복 & 성장 스토리

결제	팀원	머지	OpenAI
완전 구현	4명 협업	충돌 극복	API 연동

어떻게 극복했나

- 결제: POST /api/order/create 라우트 직접 설계, Payment 컴포넌트와 흐름 맞춤
- 관리자: JWT 미들웨어 분리 후 adminController로 책임 분산
- 머지 충돌: cart-pay.css 전량 삭제 후 Payment.css로 통합 재설계
- IP 이슈: 팀 공용 .env 파일 규칙 설정 (DB_HOST 통일)
- 브랜치 전략: front-dev / back-dev 분리 운영 → develop 머지 프로세스

△ 아쉬운 점 / 배운 점

- 초반 브랜치 전략 없이 시작 → 머지 충돌 빈발
- 공통 컴포넌트 설계 없이 각자 스타일 → CSS 충돌
- 결제 흐름 설계 문서화 없이 코딩 → 시행착오 반복
- 다음엔 API 명세서 먼저 작성 후 개발 시작 목표

회고 — FIRST 팀 프로젝트

"결제된다 무야호" 라는 커밋 메시지가 이 프로젝트의 모든 것을 담고 있습니다. 막혀 있던 결제 흐름이 뚫린 순간의 기쁨은, 그 이전까지의 좌절 — "진짜진짜진짜", "관리자용수정중인데모르겟서요엉엉" — 이 있었기 때문입니다. 처음으로 팀 협업에서 머지 충돌을 경험하고, 브랜치 전략의 필요성을 몸으로 배웠습니다. 혼자 해결할 수 없던 문제들을 팀원과 함께 풀어가며, 소통과 역할 분담이 기술만큼 중요하다는 것을 깨달았습니다.

PROJECT 06

계절

패션 쇼핑몰 · React + Node.js + Socket.io · 1인 솔로 10일

유형	기간	커밋	특이사항
1인 솔로	**26.01.15 – 01.24**	**21 commits**	**Socket.io · Nodemailer**

React 18 · Vite · Node.js · Express 5 · MySQL · JWT · Socket.io · Nodemailer · Multer · Chart.js · bcryptjs

실시간 채팅
Socket.io 1:1 고객 상담 · 관리자 채팅 관리 · 읽음 처리

등급 시스템
구매액 기반 자동 산정 (일반/브론즈/실버/골드/VIP)

비밀번호 찾기
Nodemailer Gmail SMTP 이메일 인증 링크 방식

관리자 대시보드
Chart.js 매출 분석 · 월별/시간대별/요일별 통계

주문 · 재고
DB 트랜잭션으로 주문 취소 시 재고 자동 복구

장바구니
Context API 전역 상태 · 수량 조절 · 실시간 금액 계산

19

고난과 역경

10일간의 솔로 개발 기록 — 실제 커밋이 증명하는 여정

1일	2일	3일	4일	5일	6일	7일	8일	9일	10일
		^ 3일차 두번 커밋							^ 완성!

[1일차] 학교↔집 환경 불일치

"123", "12313" 커밋 — IP 불일치로 axios baseURL 계속 수정. 집과 학교 DB 서버가 달랐음.

baseURL: http://192.168.0.219:5000 → http://192.168.0.225:5000

[3일차] 카테고리 + 채팅 동시 막힘

같은 날 커밋 2번 — "3일차 (카테고리 수정 및 채팅)" 이후 바로 다시 커밋. 한 번에 두 기능이 막힘.

380개 파일 한번에 스테이징 → 이후 단건 커밋 방식으로 전환

[2-3일차] 집에서 고친거 사이드바 고침

"집에서 고친거 사이드바 고침" — 학교에서 잘 되던 사이드바가 집 환경에서 깨짐. 머지 충돌.

ProductListPage.css 175줄 변경 + 382개 파일 재동기화 필요

[10일차] 비밀번호 찾기 + 스크롤 + IP 최종 해결

"10일차 비밀번호 찾기 구현, IP 수정(225), 채팅 스크롤 개선, vite host 설정" — 마지막 날 일괄 해결!

Nodemailer Gmail SMTP, Socket.io 스크롤 자동이동, vite.config host:true

20

결과 & 회고

계절 — 10일 만에 완성한 풀스택 쇼핑몰

10일	21개	Socket.io	100%
완성	커밋	실시간 채팅	솔로 구현

잘된 점

- 사용자·관리자 기능 전체를 1인으로 10일 완성
- Socket.io 실시간 채팅 + 읽음 처리 구현
- Nodemailer Gmail SMTP 비밀번호 찾기 구현
- Chart.js 매출/상품/고객 분석 대시보드
- DB 트랜잭션 주문 취소 → 재고 자동 복구
- 구매액 기반 자동 등급 산정 (5단계)

△ 아쉬운 점 / 성장 포인트

- 초반 IP 환경 통일 없이 시작 → 반복 수정 낭비
- 3일차 대용량 커밋 → 이후 단건 커밋으로 개선
- 타입스크립트 미적용 → 다음 프로젝트 도입 목표
- 테스트 코드 없음 → Jest + RTL 도입 계획
- CSS-in-JS 미사용 → 스타일 충돌 발생

회고 — 계절 솔로 프로젝트

"123", "집에서 고친거", "IP 수정" 같은 커밋들이 부끄럽게 보일 수도 있지만, 그것들이 바로 10일 동안 혼자서 벽에 부딪히고 해결해 나간 증거입니다. 매일 커밋이 쌓일 때마다 기능 하나씩 완성되어 갔고, 10일차 마지막 커밋 "비밀번호 찾기 구현, IP 수정(225), 채팅 스크롤 개선" 에서 모든 것이 완성되었습니다. 혼자서 프론트엔드부터 백엔드, DB, 실시간 채팅, 이메일 인증까지 전체를 구현하며 풀스택 역량의 토대를 쌓았습니다.